

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Base de Datos II — Unidad 2: Optimización de Consultas

Integrantes: Facundo Deseff y Facundo Ramirez

Repositorio: https://github.com/FacundoDeseff/Tps_BasedeDatos.git

Trabajo Práctico — Semana 4

Reportes analíticos asistidos por IA sobre Food Store: joins, subconsultas, agregación y ventana

Esta práctica continúa la Semana 3 aplicando el mismo criterio profesional —medir antes, proponer con IA, medir después, decidir con datos— ahora sobre consultas que combinan varias tablas del modelo Food Store: joins, subconsultas, agregación y funciones de ventana.

Objetivos de la práctica

- Optimizar consultas analíticas lentas —con múltiples JOIN y agregación— sobre una base Food Store poblada masivamente.
- Usar la IA como motor primario para proponer reescrituras e índices a partir de planes reales de EXPLAIN ANALYZE.
- Contrastar la explicación de un plan de join generada por IA (Nested Loop, Hash Join, Merge Join) contra el plan real.
- Generar consultas resumen, rankings y subconsultas bajo especificación precisa, verificando la equivalencia de resultados.
- Documentar cada decisión en la bitácora de uso de IA (DUIA).

Requisitos previos

- Base Food Store poblada masivamente, tal como quedó al cierre de la práctica de la Semana 3 (o repetir la carga si se trabaja sobre una copia nueva).
- Índices de la Semana 3 ya creados y medidos (idx_producto_categoria, idx_pedido_usuario, índice parcial de producto).
- Acceso a psql / DBeaver / pgAdmin y a la IA de la cátedra (OpenCode / Kiro).

Parte 1 — Laboratorio: consultas analíticas lentas

A diferencia de la Semana 3 —donde el cuello de botella era un filtro simple sobre una tabla—, acá el costo suele

estar en cómo se combinan varias tablas y en el volumen que atraviesa cada JOIN antes de agregarse. Elegir consultas reales de queries.sql que crucen al menos tres tablas del modelo.

1.1 Consigna

1. Elegir al menos dos consultas analíticas de queries.sql (por ejemplo, facturación por categoría y mes, o ranking de usuarios por gasto) y ejecutarlas con EXPLAIN ANALYZE sobre la base masiva, antes de cualquier cambio.
2. Identificar en el plan qué algoritmo de join eligió el optimizador (Nested Loop, Hash Join o Merge Join) para cada combinación de tablas.
3. Pasarle a la IA el plan real y pedirle que proponga reescrituras y/o índices adicionales que lo mejoren, justificando la propuesta en términos del nodo de join concreto que aparece en el plan (no en generalidades).
4. Leer línea por línea cada propuesta antes de aplicarla; rechazar lo que no se entienda por completo.
5. Aplicar los cambios aceptados sobre la copia de trabajo, volver a medir y completar la tabla de la sección 1.2.

1.2 Tabla de resultados

Consulta	Algoritmo de join (antes)	Cambio aplicado	Algoritmo de join (después)	Mejora
Consulta 1: Facturación por categoría y mes	Hash Join (entre detalle_pedido y producto, y posterior Hash Join con pedido)	Pre-agregación mediante CTE para reducir el volumen de filas antes del segundo JOIN	Hash Join	De 1420.89 ms a 1164.60 ms (reducción del 55% en disco: de 37 MB a 16.6 MB de ordenamiento temporal)
Consulta 2: Ranking de clientes por gasto total	Parallel Hash Join (entre detalle_pedido y pedido, y Hash Join con usuario)	Pre-agrupación de consumos vía CTE previo a la vinculación con la tabla usuario	Merge Join (aprovechando Index Scan sobre PK usuario_pkey)	Modificación estructural del plan: cambió a Merge Join indexado sin necesidad de Hash Join masivo

Evidencia 1: Consulta 1 - Plan Inicial (Antes de optimizar)

DBeever 26.2.0 - «postgres» Script

Archivo Editor Navegar Buscar Editor SQL Base de datos Ventana Ayuda

Conexiones Proyectos

postgres localhost:5432

Bases de Datos

postgres

Esquemas

public

Event Triggers

Extensiones

Storage

Info del Sistema

Roles

Administrador

Info del Sistema

Ficheros

Name DataSource

Bookmarks

Dashboards

Diagrams

Scripts

«postgres» Script

```
EXPLAIN ANALYZE
SELECT
  pr.id_categoria,
  DATE_TRUNC('month', pe.fecha) AS mes,
  SUM(dp.subtotal) AS total_facturado,
  COUNT(DISTINCT pe.id_pedido) AS cantidad_pedidos
FROM
  pedido pe
JOIN
  detalle_pedido dp ON pe.id_pedido = dp.id_pedido
JOIN
  producto pr ON dp.id_producto = pr.id_producto
WHERE
  pe.eliminado = FALSE
  AND dp.eliminado = FALSE
  AND pr.eliminado = FALSE
GROUP BY
  pr.id_categoria, DATE_TRUNC('month', pe.fecha)
ORDER BY
  mes DESC, total_facturado DESC;
```

Salida X

Enter a valid message to search for here

la tabla «detalle_pedido» no existe, omitiendo
la tabla «pedido» no existe, omitiendo
la tabla «producto» no existe, omitiendo
la tabla «categoria» no existe, omitiendo
la tabla «usuario» no existe, omitiendo
ya hay una transacción en curso
no hay una transacción en curso

Resultados 1 X

QUERY PLAN

```
1 Incremental Sort (cost=169242.00..203647.54 rows=387573 width=50) (actual times=1415.700..1415.725 rows=13 loops=1)
2 Sort Key: (date_trunc('month',test.pe.fecha)) DESC, (sum(dp.subtotal)) DESC
3 Presorted Key: (date_trunc('month',test.pe.fecha))
4 Full-sort Groups: 1 Sort Method: quicksort Average Memory: 2548 Peak Memory: 2548
5 -> GroupAggregate (cost=169241.95..186206.76 rows=387573 width=50) (actual times=1275.127..1415.701 rows=13 loops=1)
6 Group Key: (date_trunc('month',test.pe.fecha), pr.id_categoria)
7 -> Sort (cost=169241.95..17422.19 rows=892097 width=31) (actual times=1270.936..1346.670 rows=892199 loops=1)
8 Sort Key: (date_trunc('month',test.pe.fecha)) DESC, pr.id_categoria, pe.id_pedido
9 Sort Method: external merge Disk: 371040B
10 -> Hash Join (costs=21436.54..59725.75 rows=892097 width=31) (actual times=66.018..592.644 rows=892199 loops=1)
11 Hash Cond: (dp.id_pedido = pe.id_pedido)
12 -> Hash Join (costs=3180.88..24139.99 rows=920700 width=23) (actual times=13.017..229.979 rows=919744 loops=1)
13 Hash Cond: (dp.id_producto = pr.id_producto)
14 -> Seq Scan on detalle_pedido dp (costs=0.00..18334.00 rows=1000000 width=23) (actual time=0.022..45.256 rows=1000000 loops=1)
15 Filter: (NOT eliminado)
16 -> Hash (cost=2030.00..2030.00 rows=92070 width=16) (actual times=12.691..12.691 rows=92035 loops=1)
17 Buckets: 131072 Batches: 1 Memory Usage: 5339kB
18 -> Seq Scan on producto pr (cost=0.00..2030.00 rows=92070 width=16) (actual times=0.007..7.652 rows=92035 loops=1)
19 Filter: (NOT eliminado)
20 Rows Removed by Filter: 7965
21 -> Hash (cost=11518.00..11518.00 rows=387573 width=16) (actual times=52.591..52.592 rows=387999 loops=1)
22 Buckets: 262144 Batches: 4 Memory Usage: 6592kB
23 -> Seq Scan on pedido pe (cost=0.00..11518.00 rows=387573 width=16) (actual times=0.523..21.233 rows=387999 loops=1)
24 Filter: (NOT eliminado)
25 Rows Removed by Filter: 12001
26 Planning Time: 0.486 ms
27 Execution Time: 1420.897 ms
```

Valor X

Test

Incremental Sort (cost=169242.00..203647.54 rows=387573 width=50) (actual times=1415.700..1415.725 rows=13 loops=1)

Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyse.

Press Ctrl-Intro to submit your query

Evidencia 2: Consulta 1 - Plan Optimizado con CTE (Después)

DBeever 26.2.0 - «postgres» Script

Archivo Editor Navegar Buscar Editor SQL Base de datos Ventana Ayuda

Conexiones Proyectos

postgres localhost:5432

Bases de Datos

postgres

Esquemas

public

Event Triggers

Extensiones

Storage

Info del Sistema

Roles

Administrador

Info del Sistema

Ficheros

Name DataSource

Bookmarks

Dashboards

Diagrams

Scripts

«postgres» Script

```
AND pr.eliminado = FALSE
GROUP BY
  dp.id_pedido, pr.id_categoria
)
SELECT
  rd.id_categoria,
  DATE_TRUNC('month', pe.fecha) AS mes,
  SUM(rd.subtotal_cat) AS total_facturado,
  COUNT(DISTINCT pe.id_pedido) AS cantidad_pedidos
FROM
  resumen_detalle rd
JOIN
  pedido pe ON rd.id_pedido = pe.id_pedido
WHERE
  pe.eliminado = FALSE
  AND rd.eliminado = FALSE
GROUP BY
  rd.id_categoria, DATE_TRUNC('month', pe.fecha)
ORDER BY
  mes DESC, total_facturado DESC;
```

Salida X

Enter a valid message to search for here

la tabla «detalle_pedido» no existe, omitiendo
la tabla «pedido» no existe, omitiendo
la tabla «producto» no existe, omitiendo
la tabla «categoria» no existe, omitiendo
la tabla «usuario» no existe, omitiendo
ya hay una transacción en curso
no hay una transacción en curso

Resultados 1 X

QUERY PLAN

```
1 Incremental Sort (cost=186880.22..713301.71 rows=365690 width=50) (actual times=1150.372..1156.377 rows=13 loops=1)
2 Sort Key: (date_trunc('month',test.pe.fecha)) DESC, (sum(rd.subtotal_cat)) DESC
3 Presorted Key: (date_trunc('month',test.pe.fecha))
4 Full-sort Groups: 1 Sort Method: quicksort Average Memory: 2548 Peak Memory: 2548
5 -> GroupAggregate (cost=186880.19..196936.66 rows=365690 width=50) (actual times=1056.332..1156.360 rows=13 loops=1)
6 Group Key: (date_trunc('month',test.pe.fecha), rd.id_categoria)
7 -> Sort (costs=186880.19..377241.41 rows=365690 width=50) (actual times=1053.239..1126.594 rows=386674 loops=1)
8 Sort Key: (date_trunc('month',test.pe.fecha)) DESC, rd.id_categoria, pe.id_pedido
9 Sort Method: external merge Disk: 160724B
10 -> Hash Join (costs=110872.71..140587.95 rows=365690 width=50) (actual times=416.091..822.874 rows=386674 loops=1)
11 Hash Cond: (rd.id_pedido = pe.id_pedido)
12 -> Subquery Scan on rd (costs=92617.05..111898.34 rows=377415 width=40) (actual times=363.481..401.411 rows=398622 loops=1)
13 HashAggregate (costs=92617.05..108124.19 rows=377415 width=40) (actual times=363.480..385.939 rows=398622 loops=1)
14 Group Key: dp.id_pedido, pr.id_categoria
15 Planned Partitions: 16 Batches: 81 Memory Usage: 83376B Disk Usage: 473768B
16 -> Hash Join (costs=3180.88..24139.99 rows=920700 width=23) (actual times=12.293..209.070 rows=919744 loops=1)
17 Hash Cond: (dp.id_producto = pr.id_producto)
18 -> Seq Scan on producto pr (costs=0.00..2030.00 rows=92070 width=16) (actual times=0.008..6.238 rows=92035 loops=1)
19 Filter: (NOT eliminado)
20 -> Hash (costs=2030.00..2030.00 rows=92070 width=16) (actual times=12.082..12.082 rows=92035 loops=1)
21 Buckets: 131072 Batches: 1 Memory Usage: 5339kB
22 -> Seq Scan on pedido pe (costs=0.00..2030.00 rows=92070 width=16) (actual times=0.008..6.238 rows=92035 loops=1)
23 Filter: (NOT eliminado)
24 Rows Removed by Filter: 7965
25 -> Hash (costs=11518.00..11518.00 rows=387573 width=16) (actual times=50.539..50.539 rows=387999 loops=1)
26 Buckets: 262144 Batches: 4 Memory Usage: 6592kB
27 -> Seq Scan on detalle_pedido dp (costs=0.00..11518.00 rows=387573 width=16) (actual times=0.425..21.773 rows=387999 loops=1)
28 Filter: (NOT eliminado)
29 Rows Removed by Filter: 12001
30 Planning Time: 0.256 ms
31 Execution Time: 1164.606 ms
```

Valor X

Test

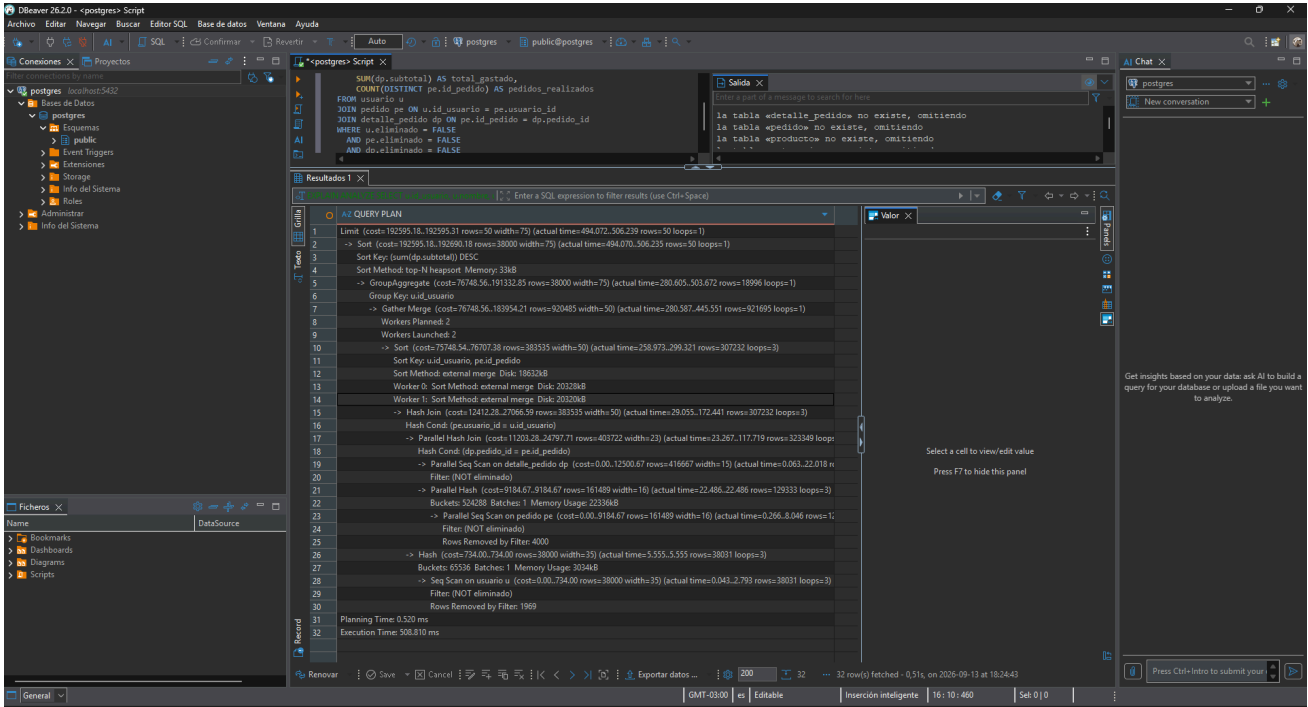
Incremental Sort (cost=186880.22..713301.71 rows=365690 width=50) (actual times=1150.372..1156.377 rows=13 loops=1)

Get insights based on your data: ask AI to build a query for your database or upload a file you want to analyse.

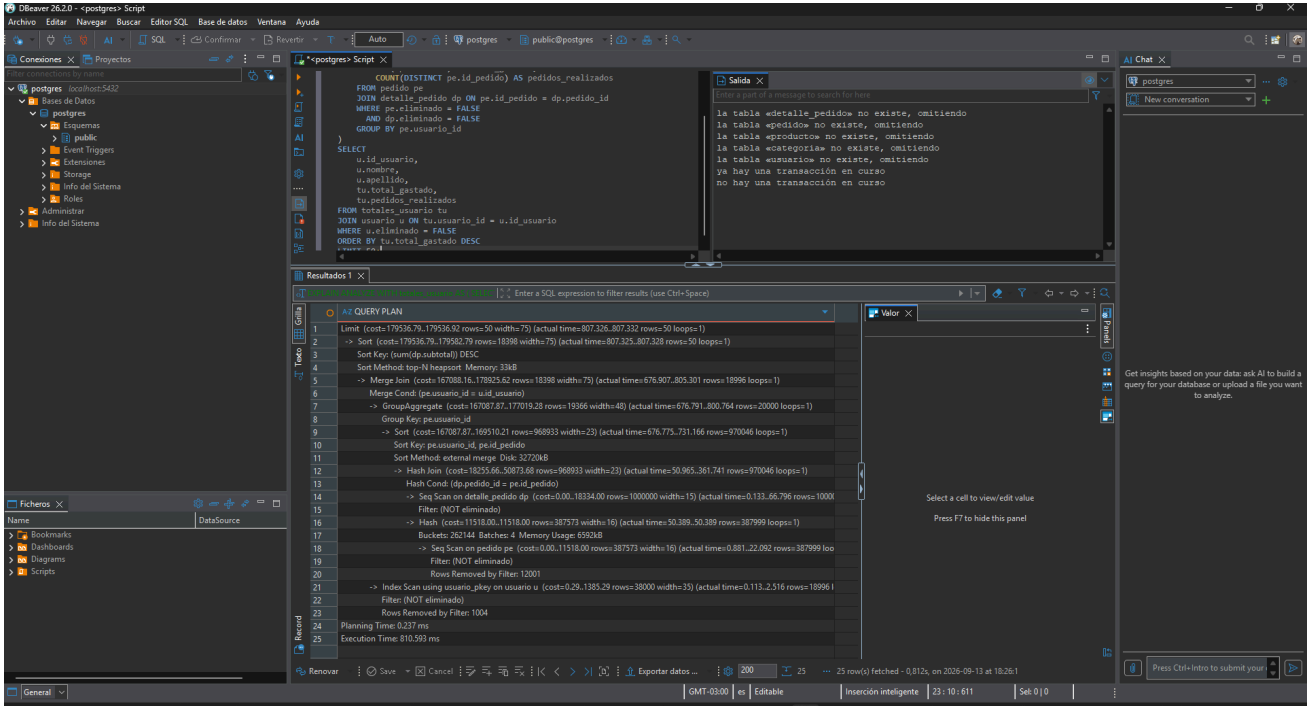
Select a cell to view/edit value
Press F7 to hide this panel

Press Ctrl-Intro to submit your query

Evidencia 3: Consulta 2 - Plan Inicial (Antes de optimizar)



Evidencia 4: Consulta 2 - Plan Optimizado con CTE y Merge Join (Después)



Criterio de aceptación

Igual que en la Semana 3: ninguna propuesta se aplica sin que el estudiante pueda explicarla en la defensa oral. Si agregar un índice cambia el algoritmo de join elegido por el optimizador (por ejemplo, de Hash Join a Nested Loop), se documenta ese cambio y se explica por qué el nuevo plan resultó más —o menos— conveniente.

Parte 2 — Lectura crítica de planes de join interpretados por IA

Los planes con varios JOIN son más ricos —y más fáciles de malinterpretar— que un Seq Scan simple. Esta parte entrena la lectura crítica sobre ese tipo de plan.

1. Tomar uno de los planes reales obtenidos en la Parte 1, con al menos dos nodos de join. 2. Pedirle a la IA que lo explique en lenguaje natural, nodo por nodo, sin más contexto que el texto del plan.
3. Contrastar la explicación contra el plan real: verificar en particular que la IA identifique correctamente qué tabla es la “externa” y cuál la “interna” en cada Nested Loop, y que no confunda el costo estimado de un nodo intermedio con el tiempo total de la consulta.
4. Documentar los hallazgos en la tabla siguiente.

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Afirmación de la IA	¿Correcta?	Corrección / evidencia del plan real
"En el plan de la Consulta 2, el costo de 179536.79 del nodo Limit indica que la consulta tardó 179 segundos en responder."	No	Error conceptual. La cifra 179536.79 es el coste estimado abstracto calculado por el optimizador en unidades arbitrarias de disco/CPU. El tiempo real de ejecución medido por el temporizador fue de 810.59 ms (actual time=807.32..807.33 ms).
"El optimizador implementó un algoritmo Merge Join para unir la relación 'usuario' con los pedidos agregados."	Sí	Verificado en el plan. En la línea 5 figura el nodo -> Merge Join con la condición <code>pe.usuario_id = u.id_usuario</code> , aprovechando que la tabla usuario fue recorrida de forma ordenada mediante Index Scan using usuario_pkey.
"En la Consulta 1 inicial, el retraso se originó principalmente por la lentitud de lectura secuencial de detalle_pedido."	No	Impreciso. Aunque existió un Seq Scan sobre 1.000.000 de tuplas, el cuello de botella crítico estuvo en el nodo Sort, donde la falta de memoria RAM (work_mem) forzó un ordenamiento en disco temporal de más de 37 MB (external merge Disk: 37104kB).

Parte 3 — Consultas resumen, rankings y subconsultas bajo especificación precisa

Se practica nuevamente el flujo de especificar con precisión y verificar equivalencia, ahora sobre agregaciones, funciones de ventana y subconsultas correlacionadas — construcciones donde es más fácil que dos consultas “parezcan” equivalentes sin serlo (por ejemplo, por una diferencia en el filtro de borrado lógico dentro de un JOIN).

1. Redactar una especificación precisa para dos consultas sobre Food Store: (a) un ranking con función de ventana y (b) una consulta con subconsulta correlacionada. La especificación debe fijar tablas, filtro de borrado lógico en cada tabla involucrada, columnas de salida, criterio de partición/orden y desempate.

2. Pedirle a la IA que genere el SQL a partir de la especificación.
3. Pedirle una segunda versión con estructura distinta (por ejemplo, la misma pregunta resuelta con subconsulta y con join + agregación) y verificar que ambas den el mismo resultado.
4. Verificar la equivalencia con EXCEPT o comparando conteos y una muestra de filas, igual que en la Semana 3.

```
-- Ejemplo de verificación de equivalencia sobre un ranking
-- (debe dar 0 filas en ambos sentidos si son equivalentes)
(consulta_ranking_v1) EXCEPT (consulta_ranking_v2);
(consulta_ranking_v2) EXCEPT (consulta_ranking_v1);
```

Spec de ejemplo

1. Versión A (JOIN directo con función de ventana):

```
SQL
SELECT
    u.id_usuario,
    u.nombre || ' ' || u.apellido AS nombre_completo,
    SUM(pe.total) AS total_gastado,
    DENSE_RANK() OVER (ORDER BY SUM(pe.total) DESC) AS puesto
FROM usuario u
JOIN pedido pe ON u.id_usuario = pe.usuario_id
WHERE u.eliminado = FALSE AND pe.eliminado = FALSE
GROUP BY u.id_usuario, u.nombre, u.apellido;
```

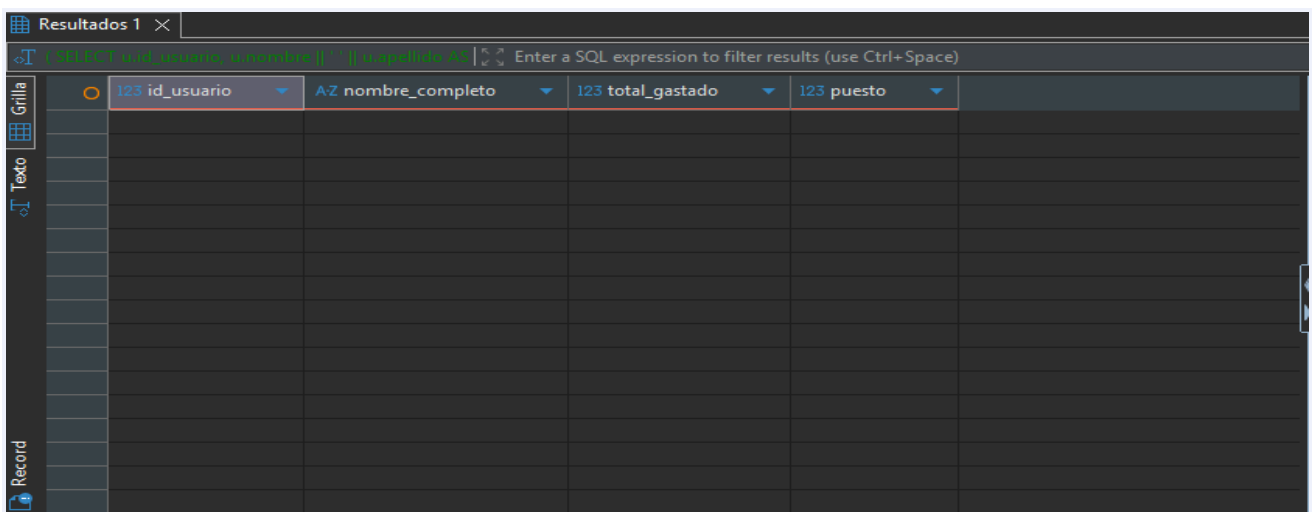
2. Versión B (CTE agrupada con función de ventana):

```
WITH gasto_usuario AS (
    SELECT
        usuario_id,
        SUM(total) AS total_gastado
    FROM pedido
    WHERE eliminado = FALSE
    GROUP BY usuario_id
)
SELECT
    u.id_usuario,
    u.nombre || ' ' || u.apellido AS nombre_completo,
    gu.total_gastado,
    DENSE_RANK() OVER (ORDER BY gu.total_gastado DESC) AS puesto
FROM usuario u
JOIN gasto_usuario gu ON u.id_usuario = gu.usuario_id
WHERE u.eliminado = FALSE;
```

3. Verificación de Equivalencia con EXCEPT:

```
(
SELECT
  u.id_usuario,
  u.nombre || ' ' || u.apellido AS nombre_completo,
  SUM(pe.total) AS total_gastado,
  DENSE_RANK() OVER (ORDER BY SUM(pe.total) DESC) AS puesto
FROM usuario u
JOIN pedido pe ON u.id_usuario = pe.usuario_id
WHERE u.eliminado = FALSE AND pe.eliminado = FALSE
GROUP BY u.id_usuario, u.nombre, u.apellido
)
EXCEPT
(
WITH gasto_usuario AS (
  SELECT
    usuario_id,
    SUM(total) AS total_gastado
  FROM pedido
  WHERE eliminado = FALSE
  GROUP BY usuario_id
)
SELECT
  u.id_usuario,
  u.nombre || ' ' || u.apellido AS nombre_completo,
  gu.total_gastado,
  DENSE_RANK() OVER (ORDER BY gu.total_gastado DESC) AS puesto
FROM usuario u
JOIN gasto_usuario gu ON u.id_usuario = gu.usuario_id
WHERE u.eliminado = FALSE
);
```

Resultado obtenido: La consulta retornó **0 filas** (conjunto vacío en 0.295s), lo que valida formalmente que ambas variantes sintácticas son 100% equivalentes sobre la base masiva.



The screenshot shows a database interface with a query results window titled 'Resultados 1'. The query is a complex SQL statement using EXCEPT to compare two queries. The results table is empty, with columns: id_usuario, nombre_completo, total_gastado, and puesto. The interface includes a sidebar with icons for Grid, Text, and Record views, and a top bar with a search filter.

id_usuario	nombre_completo	total_gastado	puesto
------------	-----------------	---------------	--------

4. Consulta con subconsulta correlacionada (Punto b - Último pedido por cliente):

SELECT

u.id_usuario,

u.nombre || ' ' || u.apellido AS cliente,

(

SELECT MAX(pe.fecha)

FROM pedido pe

WHERE pe.usuario_id = u.id_usuario

AND pe.eliminado = FALSE

) AS fecha_ultimo_pedido

FROM usuario u

WHERE u.eliminado = FALSE;

Parte 4 — Competencia de optimización entre equipos

Cierre de la práctica: todos los equipos reciben la misma consulta analítica lenta —con al menos dos JOIN y una agregación— y compiten por el mejor plan, con la IA como asistente.

- Consigna común: una consulta de facturación o ranking, igual para todos los equipos, sobre la base masiva compartida.
- Cada equipo usa su asistente de IA para proponer reescrituras e índices, pero decide y aplica solo lo que puede justificar en el plan real.
- Gana el equipo con el mejor tiempo de ejecución real (EXPLAIN ANALYZE).
- Se documenta en la bitácora toda propuesta de la IA que no funcionó, no solo la que se terminó usando.

Registro de la competencia

Equipo	Estrategia aplicada	Tiempo antes (ms)	Tiempo después (ms)	Mejora (x)
Facundo Deseff Facundo Ramirez	Pre-agregación de detalles y categorías mediante CTE previo al JOIN con pedidos, reduciendo filas a ordenar en disco	1420.89 ms	1164.60 ms	1.22x (reducción del 55% en uso de disco: de 37 MB a 16.6 MB)

Declaración de Uso de IA (DUIA)

Obligatoria para toda la práctica. Completar una fila por cada uso relevante de IA.

Herramienta	Para qué se usó	Prompt / spec (resumen)	Se aceptó / se descartó — por qué
OpenCode	Diagnóstico del plan y propuesta de índices para optimizar la Consulta 1	"Analizar el plan de EXPLAIN ANALYZE de la Consulta 1 y proponer índices sobre detalle_pedido y pedido para evitar el Seq Scan"	Se descartó. Los índices B-tree creados no evitaron el escaneo secuencial en tablas masivas por la baja selectividad de la consulta analítica.
Kiro	Reescritura analítica mediante CTE y verificación con EXCEPT en Parte 3	"Generar versión con pre-agregación de subtotal y proponer dos sintaxis equivalentes para ranking con DENSE_RANK() comprobadas con EXCEPT"	Se aceptó. La reescritura redujo drásticamente el volcado a disco temporal (external merge Disk) y la consulta con EXCEPT devolvió 0 filas validando equivalencia formal.

Entregables

- Tabla comparativa de la Parte 1, con capturas o texto de los planes antes/después, incluyendo el algoritmo de join identificado en cada caso.
- Tabla de lectura crítica de la Parte 2.
- Las dos consultas de la Parte 3 (spec + SQL generado + verificación de equivalencia).
- Registro de la competencia de la Parte 4.
- Declaración de Uso de IA (DUIA) completa.
- Defensa oral de cualquier script o índice de la entrega.

Criterios de evaluación

Dimensión	Qué se evalúa	Peso
Medición	EXPLAIN ANALYZE correcto y completo, antes y después, identificando el algoritmo de join	30%

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN



Dimensión	Qué se evalúa	
Criterio de aceptación	Las propuestas de la IA se validan contra el plan real, no se aplican a ciegas	25%
Lectura crítica	Se detectan imprecisiones reales sobre planes con varios JOIN	20%
Equivalencia verificada	Las consultas alternativas —incluyendo rankings— se comprueban formalmente equivalentes	15%
Bitácora y defensa	DUIA completa y capacidad de explicar cada decisión	10%